JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.jar

Interface Pack200.Packer

Enclosing class:

Pack200

```
public static interface Pack200.Packer
```

The packer engine applies various transformations to the input JAR file, making the pack stream highly compressible by a compressor such as gzip or zip. An instance of the engine can be obtained using `Pack200.newPacker()`. The high degree of compression is achieved by using a number of techniques described in the JSR 200 specification. Some of the techniques are sorting, re-ordering and co-location of the constant pool.

The pack engine is initialized to an initial state as described by their properties below. The initial state can be manipulated by getting the engine properties (using `properties()`) and storing the modified properties on the map. The resource files will be passed through with no changes at all. The class files will not contain identical bytes, since the unpacker is free to change minor class file features such as constant pool order. However, the class files will be semantically identical, as specified in *The Java™ Virtual Machine Specification*.

By default, the packer does not change the order of JAR elements. Also, the modification time and deflation hint of each JAR element is passed unchanged. (Any other ZIP-archive information, such as extra attributes giving Unix file permissions, are lost.)

Note that packing and unpacking a JAR will in general alter the bytewise contents of classfiles in the JAR. This means that packing and unpacking will in general invalidate any digital signatures which rely on bytewise images of JAR elements. In order both to sign and to pack a JAR, you must first pack and unpack the JAR to "normalize" it, then compute signatures on the unpacked JAR elements, and finally repack the signed JAR. Both packing steps should use precisely the same options,

and the segment limit may also need to be set to "-1", to prevent accidental variation of segment boundaries as class file sizes change slightly.

(Here's why this works: Any reordering the packer does of any classfile structures is idempotent, so the second packing does not change the orderings produced by the first packing. Also, the unpacker is guaranteed by the JSR 200 specification to produce a specific bytewise image for any given transmission ordering of archive elements.)

In order to maintain backward compatibility, the pack file's version is set to accommodate the class files present in the input JAR file. In other words, the pack file version will be the latest, if the class files are the latest and conversely the pack file version will be the oldest if the class file versions are also the oldest. For intermediate class file versions the corresponding pack file version will be used. For example: If the input JAR-files are solely comprised of 1.5 (or lesser) class files, a 1.5 compatible pack file is produced. This will also be the case for archives that have no class files. If the input JAR-files contains a 1.6 class file, then the pack file version will be set to 1.6.

Note: Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Since:

1.5

Field Summary

Fields

Modifier and Type	Field	Description
static String	CLASS_ATTRIBUTE_PFX	When concatenated with a class attribute name, indicates the format of that attribute, using the layout language specified in the JSR 200 specification.
static String	CODE_ATTRIBUTE_PFX	When concatenated with a code attribute name, indicates the format of that attribute.
static String	DEFLATE_HINT	If this property is set to **TRUE** or **FALSE**, the packer will set the deflation hint accordingly in the output archive, and will not transmit the individual deflation hints of archive elements.

static String	EFFORT	If this property is set to a single decimal digit, the packer will use the indicated amount of effort in compressing the archive.
static String	ERROR	The string "error", a possible value for certain properties.
static String	FALSE	The string "false", a possible value for certain properties.
static String	FIELD_ATTRIBUTE_PFX	When concatenated with a field attribute name, indicates the format of that attribute.
static String	KEEP	The string "keep", a possible value for certain properties.
static String	KEEP_FILE_ORDER	If this property is set to TRUE, the packer will transmit all elements in their original order within the source archive.
static String	LATEST	The string "latest", a possible value for certain properties.
static String	METHOD_ATTRIBUTE_PFX	When concatenated with a method attribute name, indicates the format of that attribute.
static String	MODIFICATION_TIME	If this property is set to the special string LATEST, the packer will attempt to determine the latest modification time, among all the available entries in the original archive or the latest modification time of all the available entries in each segment.
static String	PASS	The string "pass", a possible value for certain properties.
static String	PASS_FILE_PFX	

		Indicates that a file should be passed through bytewise, with no compression.
static String	PROGRESS	The unpacker's progress as a percentage, as periodically updated by the unpacker.
static String	SEGMENT_LIMIT	This property is a numeral giving the estimated target size N (in bytes) of each archive segment.
static String	STRIP	The string "strip", a possible value for certain properties.
static String	TRUE	The string "true", a possible value for certain properties.
static String	UNKNOWN_ATTRIBUTE	Indicates the action to take when a class-file containing an unknown attribute is encountered.

Method Summary

Deprecа

Modifier and Type	Method
default void	addPropertyChangeListener(PropertyChangeListener listen
void	pack(JarFile in, OutputStream out)

void	**pack**(**JarInputStream** in, **OutputStream** out)
SortedMap<**String**,**String**>	**properties**()
default void	**removePropertyChangeListener**(**PropertyChangeListener** lis

Field Detail

SEGMENT_LIMIT

```
static final String SEGMENT_LIMIT
```

This property is a numeral giving the estimated target size N (in bytes) of each archive segment. If a single input file requires more than N bytes, it will be given its own archive segment.

As a special case, a value of -1 will produce a single large segment with all input files, while a value of 0 will produce one segment for each class. Larger archive segments result in less fragmentation and better compression, but processing them requires more memory.

The size of each segment is estimated by counting the size of each input file to be transmitted in the segment, along with the size of its name and other transmitted properties.

The default is -1, which means the packer will always create a single segment output file. In cases where extremely large output files are generated, users are strongly encouraged to use segmenting or break up the input file into smaller JARs.

A 10Mb JAR packed without this limit will typically pack about 10% smaller, but the packer may require a larger Java heap (about ten times the segment limit).

See Also:

Constant Field Values

KEEP_FILE_ORDER

```
static final String KEEP_FILE_ORDER
```

If this property is set to TRUE, the packer will transmit all elements in their original order within the source archive.

If it is set to FALSE, the packer may reorder elements, and also remove JAR directory entries, which carry no useful information for Java applications. (Typically this enables better compression.)

The default is TRUE, which preserves the input information, but may cause the transmitted archive to be larger than necessary.

See Also:

Constant Field Values

EFFORT

```
static final String EFFORT
```

If this property is set to a single decimal digit, the packer will use the indicated amount of effort in compressing the archive. Level 1 may produce somewhat larger size and faster compression speed, while level 9 will take much longer but may produce better compression.

The special value 0 instructs the packer to copy through the original JAR file directly, with no compression. The JSR 200 standard requires any unpacker to understand this special case as a pass-through of the entire archive.

The default is 5, investing a modest amount of time to produce reasonable compression.

See Also:

Constant Field Values

DEFLATE_HINT

```
static final String DEFLATE_HINT
```

If this property is set to TRUE or FALSE, the packer will set the deflation hint accordingly in the output archive, and will not transmit the individual deflation hints of archive elements.

If this property is set to the special string KEEP, the packer will attempt to determine an independent deflation hint for each available element of the input archive, and transmit this hint separately.

The default is KEEP, which preserves the input information, but may cause the transmitted archive to be larger than necessary.

It is up to the unpacker implementation to take action upon the hint to suitably compress the elements of the resulting unpacked jar.

The deflation hint of a ZIP or JAR element indicates whether the element was deflated or stored directly.

See Also:

Constant Field Values

MODIFICATION_TIME

```
static final String MODIFICATION_TIME
```

If this property is set to the special string LATEST, the packer will attempt to determine the latest modification time, among all the available entries in the original archive or the latest modification time of all the available entries in each segment. This single value will be transmitted as part of the segment and applied to all the entries in each segment, SEGMENT_LIMIT.

This can marginally decrease the transmitted size of the archive, at the expense of setting all installed files to a single date.

If this property is set to the special string KEEP, the packer transmits a separate modification time for each input element.

The default is KEEP, which preserves the input information, but may cause the transmitted archive to be larger than necessary.

It is up to the unpacker implementation to take action to suitably set the modification time of each element of its output file.

See Also:

SEGMENT_LIMIT, Constant Field Values

PASS_FILE_PFX

```
static final String PASS_FILE_PFX
```

Indicates that a file should be passed through bytewise, with no compression. Multiple files may be specified by specifying additional properties with distinct strings appended, to make a family of properties with the common prefix.

There is no pathname transformation, except that the system file separator is replaced by the JAR file separator '/'.

The resulting file names must match exactly as strings with their occurrences in the JAR file.

If a property value is a directory name, all files under that directory will be passed also.

Examples:

```
Map p = packer.properties();
p.put(PASS_FILE_PFX+0, "mutants/Rogue.class");
p.put(PASS_FILE_PFX+1, "mutants/Wolverine.class");
p.put(PASS_FILE_PFX+2, "mutants/Storm.class");
# Pass all files in an entire directory hierarchy:
p.put(PASS_FILE_PFX+3, "police/");
```

See Also:

Constant Field Values

UNKNOWN_ATTRIBUTE

`static final String UNKNOWN_ATTRIBUTE`

Indicates the action to take when a class-file containing an unknown attribute is encountered. Possible values are the strings `ERROR`, `STRIP`, and `PASS`.

The string `ERROR` means that the pack operation as a whole will fail, with an exception of type `IOException`. The string `STRIP` means that the attribute will be dropped. The string `PASS` means that the whole class-file will be passed through (as if it were a resource file) without compression, with a suitable warning. This is the default value for this property.

Examples:

```
Map p = pack200.getProperties();
p.put(UNKNOWN_ATTRIBUTE, ERROR);
p.put(UNKNOWN_ATTRIBUTE, STRIP);
p.put(UNKNOWN_ATTRIBUTE, PASS);
```

See Also:

Constant Field Values

CLASS_ATTRIBUTE_PFX

`static final String CLASS_ATTRIBUTE_PFX`

When concatenated with a class attribute name, indicates the format of that attribute, using the layout language specified in the JSR 200 specification.

For example, the effect of this option is built in:
`pack.class.attribute.SourceFile=RUH`.

The special strings `ERROR`, `STRIP`, and `PASS` are also allowed, with the same meaning as `UNKNOWN_ATTRIBUTE`. This provides a way for users to request that specific attributes be refused, stripped, or passed bitwise (with no class compression).

Code like this might be used to support attributes for JCOV:

```
Map p = packer.properties();
p.put(CODE_ATTRIBUTE_PFX+"CoverageTable",       "NH[PHHII]");
p.put(CODE_ATTRIBUTE_PFX+"CharacterRangeTable", "NH[PHPOHIIH]");
p.put(CLASS_ATTRIBUTE_PFX+"SourceID",           "RUH");
p.put(CLASS_ATTRIBUTE_PFX+"CompilationID",      "RUH");
```

Code like this might be used to strip debugging attributes:

```
Map p = packer.properties();
p.put(CODE_ATTRIBUTE_PFX+"LineNumberTable",    STRIP);
p.put(CODE_ATTRIBUTE_PFX+"LocalVariableTable", STRIP);
p.put(CLASS_ATTRIBUTE_PFX+"SourceFile",        STRIP);
```

See Also:

`Constant Field Values`

FIELD_ATTRIBUTE_PFX

`static final String FIELD_ATTRIBUTE_PFX`

When concatenated with a field attribute name, indicates the format of that attribute. For example, the effect of this option is built in:
`pack.field.attribute.Deprecated=`. The special strings `ERROR`, `STRIP`, and `PASS` are also allowed.

See Also:

`CLASS_ATTRIBUTE_PFX`, `Constant Field Values`

METHOD_ATTRIBUTE_PFX

`static final String METHOD_ATTRIBUTE_PFX`

When concatenated with a method attribute name, indicates the format of that attribute. For example, the effect of this option is built in: `pack.method.attribute.Exceptions=NH[RCH]`. The special strings `ERROR`, `STRIP`, and `PASS` are also allowed.

See Also:

`CLASS_ATTRIBUTE_PFX`, `Constant Field Values`

CODE_ATTRIBUTE_PFX

`static final String CODE_ATTRIBUTE_PFX`

When concatenated with a code attribute name, indicates the format of that attribute. For example, the effect of this option is built in: `pack.code.attribute.LocalVariableTable=NH[PHOHRUHRSHH]`. The special strings `ERROR`, `STRIP`, and `PASS` are also allowed.

See Also:

`CLASS_ATTRIBUTE_PFX`, `Constant Field Values`

PROGRESS

`static final String PROGRESS`

The unpacker's progress as a percentage, as periodically updated by the unpacker. Values of 0 - 100 are normal, and -1 indicates a stall. Progress can be monitored by polling the value of this property.

At a minimum, the unpacker must set progress to 0 at the beginning of a packing operation, and to 100 at the end.

See Also:

`Constant Field Values`

KEEP

```
static final String KEEP
```

The string "keep", a possible value for certain properties.

See Also:

DEFLATE_HINT, MODIFICATION_TIME, Constant Field Values

PASS

```
static final String PASS
```

The string "pass", a possible value for certain properties.

See Also:

UNKNOWN_ATTRIBUTE, CLASS_ATTRIBUTE_PFX, FIELD_ATTRIBUTE_PFX, METHOD_ATTRIBUTE_PFX, CODE_ATTRIBUTE_PFX, Constant Field Values

STRIP

```
static final String STRIP
```

The string "strip", a possible value for certain properties.

See Also:

UNKNOWN_ATTRIBUTE, CLASS_ATTRIBUTE_PFX, FIELD_ATTRIBUTE_PFX, METHOD_ATTRIBUTE_PFX, CODE_ATTRIBUTE_PFX, Constant Field Values

ERROR

```
static final String ERROR
```

The string "error", a possible value for certain properties.

See Also:

UNKNOWN_ATTRIBUTE, CLASS_ATTRIBUTE_PFX, FIELD_ATTRIBUTE_PFX, METHOD_ATTRIBUTE_PFX, CODE_ATTRIBUTE_PFX, Constant Field Values

TRUE

```
static final String TRUE
```

The string "true", a possible value for certain properties.

See Also:

KEEP_FILE_ORDER, DEFLATE_HINT, Constant Field Values

FALSE

```
static final String FALSE
```

The string "false", a possible value for certain properties.

See Also:

KEEP_FILE_ORDER, DEFLATE_HINT, Constant Field Values

LATEST

```
static final String LATEST
```

The string "latest", a possible value for certain properties.

See Also:

MODIFICATION_TIME, Constant Field Values

Method Detail

properties

```
SortedMap<String,String> properties()
```

Get the set of this engine's properties. This set is a "live view", so that changing its contents immediately affects the Packer engine, and changes from the engine (such as progress indications) are immediately visible in the map.

The property map may contain pre-defined implementation specific and default properties. Users are encouraged to read the information and fully understand the implications, before modifying pre-existing properties.

Implementation specific properties are prefixed with a package name associated with the implementor, beginning with `com.` or a similar prefix. All property names beginning with `pack.` and `unpack.` are reserved for use by this API.

Unknown properties may be ignored or rejected with an unspecified error, and invalid entries may cause an unspecified error to be thrown.

The returned map implements all optional SortedMap operations

Returns:

A sorted association of property key strings to property values.

pack

```
void pack(JarFile in,
          OutputStream out)
   throws IOException
```

Takes a JarFile and converts it into a Pack200 archive.

Closes its input but not its output. (Pack200 archives are appendable.)

Parameters:

in - a JarFile

out - an OutputStream

Throws:

IOException - if an error is encountered.

pack

```
void pack(JarInputStream in,
          OutputStream out)
   throws IOException
```

Takes a JarInputStream and converts it into a Pack200 archive.

Closes its input but not its output. (Pack200 archives are appendable.)

The modification time and deflation hint attributes are not available, for the JAR manifest file and its containing directory.

Parameters:

in - a JarInputStream

out - an OutputStream

Throws:

IOException - if an error is encountered.

See Also:

MODIFICATION_TIME, DEFLATE_HINT

addPropertyChangeListener

```
@Deprecated
default void addPropertyChangeListener(PropertyChangeListener listener)
```

Deprecated. *The dependency on `PropertyChangeListener` creates a significant impediment to future modularization of the Java platform. This method will be removed in a future release. Applications that need to monitor progress of the packer can poll the value of the `PROGRESS` property instead.*

Registers a listener for PropertyChange events on the properties map. This is typically used by applications to update a progress bar.

The default implementation of this method does nothing and has no side-effects.

WARNING: This method is omitted from the interface declaration in all subset Profiles of Java SE that do not include the `java.beans` package.

Parameters:

`listener - An object to be invoked when a property is changed.`

See Also:

`properties()`, `PROGRESS`

removePropertyChangeListener

```
@Deprecated
default void removePropertyChangeListener(PropertyChangeListener listener)
```

Deprecated. *The dependency on `PropertyChangeListener` creates a significant impediment to future modularization of the Java platform. This method will be removed in a future release.*

Remove a listener for PropertyChange events, added by the `addPropertyChangeListener(java.beans.PropertyChangeListener)`.

The default implementation of this method does nothing and has no side-effects.

WARNING: This method is omitted from the interface declaration in all subset Profiles of Java SE that do not include the `java.beans` package.

Parameters:

`listener - The PropertyChange listener to be removed.`

See Also:

```
addPropertyChangeListener(java.beans.PropertyChangeListener)
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.